TECHNICAL ARCHITECTURE DOCUMENT

RIGACAP

System Architecture & Technical Audit

Comprehensive technical documentation covering infrastructure, database schema, API surface, security architecture, deployment pipeline, and operational characteristics.

Confidential — April 2026

v3.0.0 · 95 endpoints · 18 tables · 15K+ LOC

Table of Contents

Infrastructure

1. System Overview & Architecture 3
2. AWS Infrastructure (Terraform) 3
3. CDN, DNS & SSL 4

Data Layer

4. Database Schema (PostgreSQL 15) 5
5. Data Pipeline & Caching 7

Application

6. Backend Services (FastAPI) 8
7. API Surface (95 Endpoints) 9
8. Frontend Architecture (React) 11

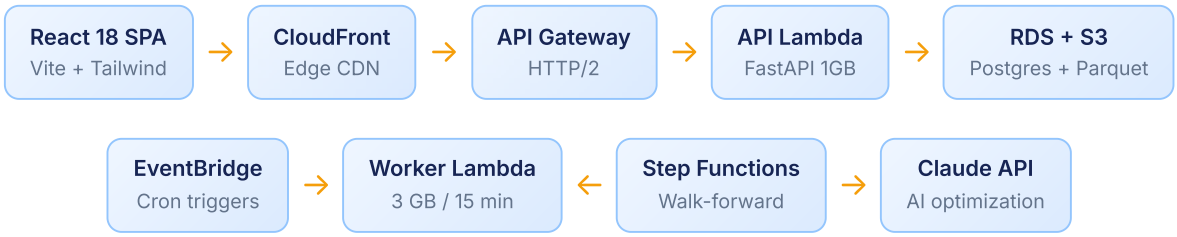
Operations

9. Security Architecture 12
10. CI/CD Pipeline 13
11. Monitoring & Performance 14
12. Dependencies & Versions 14

95	18	22+	15K+
API ENDPOINTS	DB TABLES	BACKEND SERVICES	LINES OF CODE

1. System Overview

RigaCap is a serverless, event-driven trading platform built on AWS. The system ingests market data from 4,000+ US equities via a dual-source pipeline (Alpaca SIP primary + yfinance fallback), generates AI-powered trading signals, and delivers them to subscribers via a React dashboard and automated email pipeline. The backend runs on two specialized AWS Lambda functions (API + Worker) from a single container image, using FastAPI + Mangum for ASGI compatibility.



2. AWS Infrastructure

All infrastructure is managed via Terraform (`infrastructure/terraform/main.tf`). Resources are deployed in `us-east-1` .

Compute

RESOURCE	TYPE	CONFIGURATION
API Lambda	`rigacap-prod-api`	Container image (ECR), 1 GB RAM, 30s timeout, HTTP requests only
Worker Lambda	`rigacap-prod-worker`	Same image, 3 GB RAM, 900s timeout, EventBridge jobs + background tasks
API Gateway	HTTP API (v2)	Custom domain: `api.rigacap.com` , CORS configured, HTTP/2
ECR Repository	`rigacap-prod-api`	Mutable tags, scan-on-push, keep last 5 images
Step Functions	`rigacap-prod-walk-forward`	Orchestrates Lambda for multi-period walk-forward simulations

Storage

BUCKET	ACCESS	PURPOSE
`rigacap-prod-frontend-*`	Public (CloudFront origin)	React SPA static files (~1-2MB)
`rigacap-prod-price-data-*`	Private (Lambda IAM only)	Parquet price cache, chart cards, social images (50-100MB)

Database

RESOURCE	CONFIGURATION
RDS PostgreSQL 15	`db.t3.micro`, 20GB gp2, deletion protection enabled, asyncpg driver
Connection Pool	Pool size: 5, max overflow: 10, connect timeout: 3s, query timeout: 5s

Scheduled Events (EventBridge) — 14 Rules

RULE	SCHEDULE (UTC / ET)	DESCRIPTION
Market Scanner	`cron(20 21 ? * MON-FRI *)` — 4:20 PM ET	Full scan + dashboard + signals (20 min post-close for Alpaca settlement)
Double Signal Alerts	`cron(0 22 ? * MON-FRI *)` — 5:00 PM ET	Multi-confirmation signal email alerts
Daily Email Digest	`cron(0 23 ? * MON-FRI *)` — 6:00 PM ET	Subscriber daily digest email
Strategy Analysis	`cron(30 23 ? * FRI *)` — Fri 6:30 PM ET	Weekly strategy auto-analysis
Nightly Walk-Forward	`cron(0 1 ? * TUE-SAT *)` — 8:00 PM ET	Walk-forward simulation + social post generation
AI Social Content	`cron(0 2 ? * TUE-SAT *)` — 9:00 PM ET	Generate AI social posts from live portfolio trades
Intraday Monitor	`cron(0/5 14-20 ? * MON-FRI *)` — every 5 min, market hours	Position monitoring + trailing stop checks
Ticker Health	`cron(0 12 ? * MON-FRI *)` — 7:00 AM ET	Pre-market ticker health check
Pipeline Health	`cron(30 12 * * ? *)` — 7:30 AM ET daily	17-check pipeline health report
Onboarding Drip	`cron(0 15 * * ? *)` — 10:00 AM ET daily	5-step new subscriber drip sequence

RULE	SCHEDULE (UTC / ET)	DESCRIPTION
Publish Posts	`cron(0/15 * * * ? *)` — every 15 min	Auto-publish approved scheduled social posts
Post Notifications	`cron(0 * * * ? *)` — hourly	T-24h/T-1h admin notifications for upcoming posts
Pickle Rebuild	`cron(0 1 ? * SUN *)` — Sat 8:00 PM ET	Weekly full pickle rebuild (catches new universe symbols)
Lambda Warmer	`rate(5 minutes)`	Keep Worker Lambda warm

3. CDN, DNS & SSL

CloudFront Distribution

SETTING	VALUE
Domains	`rigacap.com`, `www.rigacap.com`
Origin	S3 website endpoint (HTTP-only)
Protocol	Redirect HTTP → HTTPS
Price Class	PriceClass_100 (US, Canada, Europe)
Default TTL	1 hour (max 1 day)
SPA Routing	Custom error: 404 → `/index.html` (200)
SSL Certificate	ACM: `rigacap.com` + `*.rigacap.com` (DNS validated)

Route53 DNS

RECORD	TYPE	TARGET
`rigacap.com`	A (alias)	CloudFront distribution
`www.rigacap.com`	A (alias)	CloudFront distribution
`api.rigacap.com`	A (alias)	API Gateway custom domain

4. Database Schema

PostgreSQL 15 with async driver (`asyncpg`) via SQLAlchemy 2.0 async sessions. All tables use auto-increment integer primary keys except `users` and `subscriptions` (UUID).

Core Data Tables

stock_data

Daily OHLCV price data with computed indicators. One row per symbol per trading day.

```
id INTEGER PK                          symbol VARCHAR(10) IDX
date DATETIME IDX                      open, high, low, close FLOAT
volume FLOAT                           dwap, ma_50, ma_200 FLOAT
```

signals

Trading signals generated by the scanner. Status lifecycle: active → executed/expired.

- `id` INTEGER PK
- `signal_type` VARCHAR(10) BUY/SELL
- `volume, volume_ratio` FLOAT
- `is_strong` BOOLEAN
- `created_at, expires_at` DATETIME
- `symbol` VARCHAR(10) IDX
- `price, dwap, pct_above_dwap` FLOAT
- `stop_loss, profit_target` FLOAT
- `status` VARCHAR(20)

positions

Open/closed portfolio positions with trailing stop tracking.

- `id` INTEGER PK
- `symbol` VARCHAR(10) IDX
- `shares, stop_loss` FLOAT
- `signal_id` INTEGER FK→signals
- `user_id` UUID FK→users
- `entry_date, entry_price` DATETIME, FLOAT
- `profit_target, highest_price` FLOAT
- `status` VARCHAR(20) open/closed

trades

Completed trade records with P&L tracking.

- `id` INTEGER PK
- `position_id` INTEGER FK→positions
- `entry_date, exit_date` DATETIME
- `pnl, pnl_pct` FLOAT
- `user_id` UUID FK→users
- `symbol` VARCHAR(10) IDX
- `entry_price, exit_price` FLOAT
- `exit_reason` VARCHAR(50)

Authentication & Subscriptions

users

User accounts with multi-provider OAuth support. UUID primary keys.

- `id` UUID PK
- `password_hash` VARCHAR(255) NULLABLE
- `role` VARCHAR(20) admin/user
- `stripe_customer_id` VARCHAR(255) UNIQUE
- `created_at, last_login` DATETIME
- `email` VARCHAR(255) UNIQUE IDX
- `name` VARCHAR(255)
- `google_id, apple_id` VARCHAR(255) UNIQUE
- `is_active` BOOLEAN

subscriptions

Stripe-managed subscription state. Status: trial/active/canceled/expired/past_due.

- `id` UUID PK
- `status` VARCHAR(20)
- `stripe_price_id` VARCHAR(255)
- `current_period_start/end` DATETIME
- `user_id` UUID FK→users UNIQUE
- `stripe_subscription_id` VARCHAR(255)
- `trial_start, trial_end` DATETIME
- `cancel_at_period_end` BOOLEAN

Strategy Management

strategy_definitions

Trading strategy configurations. Types: dwap, momentum, ensemble.

- `id` INTEGER PK
- `strategy_type` VARCHAR(50)
- `is_active` BOOLEAN IDX
- `is_custom` BOOLEAN
- `name` VARCHAR(100) UNIQUE
- `parameters` TEXT (JSON)
- `source` VARCHAR(50) manual/ai

strategy_evaluations

Performance evaluations for strategies

- `strategy_id` FK→strategy_definitions
- `total_return_pct, sharpe_ratio` FLOAT
- `max_drawdown_pct, win_rate` FLOAT
- `recommendation_score` FLOAT 0-100

strategy_switch_history

Audit trail for strategy switches

- `from/to_strategy_id` FK→strategy_definitions
- `trigger` manual/auto_scheduled
- `score_before, score_after` FLOAT
- `reason` TEXT

Simulation & Backtesting

walk_forward_simulations

Walk-forward simulation runs with aggregate results. Trades stored as JSON to avoid 256KB Step Functions state limit.

id INTEGER PK
reoptimization_frequency VARCHAR(20)
max_drawdown_pct FLOAT
equity_curve_json TEXT (JSON)
is_daily_cache BOOLEAN IDX
start_date, end_date DATETIME
total_return_pct, sharpe_ratio FLOAT
trades_json TEXT (JSON array)
status pending/completed/failed

social_posts

AI-generated social content with approval workflow. Status: draft → approved → scheduled → posted/canceled.

id INTEGER PK
platform twitter/instagram/threads
text_content, hashtags TEXT
scheduled_for, posted_at DATETIME
ai_model, ai_prompt_hash VARCHAR
source_trade_json TEXT
reply_to_thread_id VARCHAR(50)
reply_to_instagram_media_id VARCHAR(50)
post_type trade_result/missed_opp/recap/contextual_reply/instagram_comment_reply
status VARCHAR(20)
image_s3_key VARCHAR(500)
ai_generated BOOLEAN
news_context_json TEXT
reply_to_tweet_id VARCHAR(50)
reply_to_instagram_comment_id VARCHAR(50)
reply_to_username VARCHAR(100)

Model Portfolio & Intelligence Tables

model_positions

Model portfolio positions (live, walkforward, ghost variants). Tracks entry/exit with signal replay data.

id INTEGER PK
symbol VARCHAR(10) IDX
shares, cost_basis FLOAT
exit_reason, status VARCHAR
autopsy_json TEXT (JSON) — AI trade post-mortem
portfolio_type VARCHAR(20) IDX
entry_date, entry_price DATETIME, FLOAT
highest_price, exit_price FLOAT
signal_data_json TEXT (JSON)

regime_forecast_snapshots

Daily regime forecast persistence for historical accuracy tracking and heatmap visualization.

`id` INTEGER PK

`snapshot_date` DATETIME UNIQUE IDX

`current_regime` VARCHAR(30)

`probabilities_json` TEXT (JSON) — 7 regime probabilities

`outlook` stable/deteriorating/improving

`recommended_action` VARCHAR(30)

`spy_close, vix_close` FLOAT

push_tokens

Mobile app push notification tokens for Expo push service.

`id` INTEGER PK

`user_id` INTEGER FK → users

`token` VARCHAR UNIQUE

`platform` VARCHAR (ios/android)

`created_at` DATETIME

regime_history

Weekly regime classifications for chart band overlays on equity curves.

`id` INTEGER PK

`week_start` DATE UNIQUE IDX

`regime` VARCHAR(30)

`spy_close, vix_close` FLOAT

5. Data Pipeline & Caching

Dual-Source Ingestion

Alpaca SIP (primary) + yfinance (fallback). Daily download of 252-day rolling OHLCV for 4,000+ symbols. Auto-failover with health tracking. Index symbols (^VIX, ^GSPC) always route to yfinance.

S3 Pickle Cache

Price data persisted to S3 as gzipped pickle (~179 MB compressed, ~2+ GB uncompressed, 7,381 symbols). Worker Lambda loads on cold start. Weekly rebuild via EventBridge.

In-Memory Cache

ScannerService maintains `data_cache: Dict[str, DataFrame]` in Lambda memory. 600-800MB typical footprint.

CDN Pre-computation

Dashboard JSON exported to S3 daily. CloudFront serves with 1-hour TTL for zero-latency dashboard loads.

6. Backend Services

29 singleton service classes comprising 20,000+ lines of Python. All services are instantiated at module level and imported globally. The Lambda function uses Mangum for ASGI-to-Lambda translation.

SERVICE	LOC	RESPONSIBILITY
ScannerService	940	Market scan, signal generation, data cache management
BacktesterService	1,255	Historical backtesting, portfolio simulation, performance metrics
WalkForwardService	1,907	Step Functions orchestration, period-by-period evaluation, AI optimization
EmailService	2,358	Subscriber emails: welcome, daily digest, sell alerts, password reset
SchedulerService	1,454	EventBridge-triggered background jobs, nightly digest, social post scheduling
ModelPortfolioService	1,200	Dual live/WF portfolio tracking, ghost portfolios, what-if calculator
MarketRegimeService	882	7-regime market classification using SPY indicators
DataExportService	827	S3 data export/import, CDN pre-computation, parquet persistence
SocialContentService	759	Template-based post generation (fallback for AI service)
AutoSwitchService	683	Automated strategy switching with scoring and cooldown

SERVICE	LOC	RESPONSIBILITY
StockUniverseService	461	Stock universe management, sector classification, ETF filtering
StrategyAnalyzerService	437	Multi-strategy comparison and performance attribution
StrategyGeneratorService	434	Optuna-based parameter optimization, AI strategy generation
ChartCardGenerator	346	1080×1350 (4:5) matplotlib chart cards with brand styling
PostSchedulerService	292	Post scheduling, notification pipeline, cancel mechanism
AIContentService	287	Claude API integration for AI-powered content generation
SocialPostingService	420	Twitter API v2 + Instagram Graph API + Threads API publishing
ReplyScannerService	650	Multi-platform contextual reply scanner (Twitter + Threads)
InstagramCommentService	200	Instagram comment monitoring + Claude-powered reply generation
TradeAutopsyService	175	Claude-powered forensic post-mortem analysis for closed trades
RegimeForecastService	185	Daily regime forecast storage, accuracy tracking, heatmap data
HealthMonitorService	920	Pipeline health checks (S3 freshness, CW metrics, DB counts), daily report emails
PushNotificationService	150	Expo push notifications for mobile app (sell alerts, signals)
EnsembleSignalService	200	Persist ensemble signals to DB for audit trail + email consistency
DualSourceProvider	500	Alpaca SIP (primary) + yfinance (fallback) with auto-failover and health tracking

Lambda Cold Start Mitigation: Database connections are initialized lazily on first request (Lambda init has a 10-second timeout). An EventBridge rule pings `/health` every 5 minutes to keep the Lambda warm. Post-deploy, `/api/warmup` preloads S3 data into memory.

7. API Surface (120+ Endpoints)

Authentication (`/auth` + `/auth/2fa`) — 19 endpoints

METHOD	PATH	DESCRIPTION
`POST`	/auth/register	Email/password signup with Turnstile verification
`POST`	/auth/login	Email/password login, returns JWT access + refresh tokens
`POST`	/auth/refresh	Refresh access token using refresh token
`GET`	/auth/me	Current user profile
`POST`	/auth/google	Google OAuth signup/login
`POST`	/auth/apple	Apple Sign In signup/login
`POST`	/auth/forgot-password	Send password reset email
`POST`	/auth/reset-password	Reset password with secure token
`POST`	/auth/logout	Invalidate current tokens
`PATCH`	/auth/me/email-preferences	Update email notification preferences
`GET`	/auth/email-preferences	Get current email preferences
`POST`	/auth/unsubscribe	One-click email unsubscribe (RFC 8058)
`GET`	/auth/referral	Get/create referral code and stats
`POST`	/auth/2fa/setup	Generate TOTP secret + QR code
`POST`	/auth/2fa/confirm-setup	Confirm TOTP setup with verification code
`POST`	/auth/2fa/verify	Verify 2FA code during login
`POST`	/auth/2fa/disable	Disable 2FA for account
`GET`	/auth/2fa/status	Check if 2FA is enabled
`POST`	/auth/2fa/regenerate-backup-codes	Generate new backup codes

Signals (`/signals`) — 14 endpoints

METHOD	PATH	AUTH
`GET`	/signals/scan	Subscriber (runs full market scan)
`GET`	/signals/latest	Subscriber
`GET`	/signals/dashboard	Subscriber (comprehensive data)
`GET`	/signals/momentum-rankings	Subscriber
`GET`	/signals/missed	Subscriber (missed opportunities)
`GET`	/signals/double-signals	Subscriber (multi-confirmation)
`GET`	/signals/watchlist	Subscriber
`GET`	/signals/approaching-trigger	Subscriber
`GET`	/signals/symbol/{symbol}	Subscriber
`GET`	/signals/info/{symbol}	Subscriber
`GET`	/signals/cdn-url	Subscriber
`GET`	/signals/memory-scan	Subscriber
`POST`	/signals/backfill	Admin
`GET`	/signals/simulate-intraday-crossover	Admin

Billing (`/billing`) — 5 endpoints

METHOD	PATH	DESCRIPTION
`POST`	/billing/create-checkout	Create Stripe Checkout session (7-day trial, CC required)
`POST`	/billing/portal	Create Stripe Customer Portal session
`GET`	/billing/subscription	Get current subscription status
`POST`	/billing/sync	Sync subscription state from Stripe
`POST`	/billing/webhook	Stripe webhook receiver (signature verified)

Social (`/social`) — 17 endpoints

METHOD	PATH	DESCRIPTION
`GET`	/social/posts	List posts (filter by status, type, platform)
`GET`	/social/posts/{id}	Get single post details

METHOD	PATH	DESCRIPTION
`POST`	/social/posts/compose	Create new draft post
`POST`	/social/posts/{id}/approve	Approve for publishing
`POST`	/social/posts/{id}/reject	Reject with reason
`GET`	/social/posts/{id}/preview	Preview rendered post
`POST`	/social/posts/{id}/publish	Publish immediately
`POST`	/social/posts/{id}/schedule	Schedule for future date
`POST`	/social/posts/{id}/cancel	Cancel scheduled post
`POST`	/social/posts/{id}/regenerate	Re-generate post content
`POST`	/social/posts/{id}/regenerate-ai	Re-generate via Claude API
`POST`	/social/posts/{id}/edit	Edit draft content
`DELETE`	/social/posts/{id}	Delete draft
`GET`	/social/posts/{id}/cancel-email	One-click cancel via JWT link
`GET`	/social/posts/{id}/approve-email	One-click approve via JWT link
`POST`	/social/generate-chart/{id}	Generate chart card image
`GET`	/social/stats	Publishing statistics

Admin (`/admin`) — 50+ endpoints

All admin endpoints require `role=admin` AND email in `ADMIN_EMAILS` allowlist. Categories include: strategy management (20+), user management (6), market regime analysis (4), walk-forward simulation (8), backtesting (2), data debugging (2), model portfolio (10), trade autopsies (2), ghost comparison (1), regime forecast (3), what-if calculator (1).

8. Frontend Architecture

React 18	Vite 5	Tailwind 3
UI FRAMEWORK	BUILD TOOL	CSS FRAMEWORK

Component Architecture

COMPONENT	SIZE	RESPONSIBILITY
App.jsx	171 KB	Main dashboard: portfolio, signals, charts, auth, time-travel (admin)
WalkForwardSimulator.jsx	67 KB	Configure/run simulations, equity curves, trade analysis
SocialTab.jsx	55 KB	Post queue, AI generation, scheduling, chart cards, publishing
FlexibleBacktest.jsx	39 KB	Backtest config, strategy comparison, exit testing
AdminDashboard.jsx	42 KB	Strategy/user management, model portfolio, ghost comparison, regime forecast, what-if calculator, trade autopsies
LandingPage.jsx	30 KB	Marketing: hero, features, pricing, FAQ, social proof
StrategyEditor.jsx	16 KB	Custom strategy creation and editing
AutoSwitchConfig.jsx	15 KB	Auto-switch rule configuration
LoginModal.jsx	14 KB	Multi-provider auth: email, Google, Apple, Turnstile
StrategyGenerator.jsx	12 KB	AI strategy generation interface
LegalPages.jsx	11 KB	Terms, Privacy Policy, Financial Disclaimer
PasswordReset.jsx	9 KB	Forgot/reset password flow
SubscriptionBanner.jsx	9 KB	Trial countdown, upgrade CTA
TrackRecordChart.jsx	8 KB	Walk-forward equity curve with regime overlays

COMPONENT	SIZE	RESPONSIBILITY
TwoFactorSettings.jsx	7 KB	TOTP 2FA setup, backup codes, enable/disable
CookieConsent.jsx	3 KB	GDPR cookie consent banner for GA4

Key Frontend Libraries

- **Recharts** — Dashboard charts, equity curves
- **Axios** — HTTP client with JWT interceptor
- **React Router v6** — SPA routing
- **Lucide React** — SVG icon library
- **react-hot-toast** — Notification system
- **date-fns** — Date formatting/manipulation
- **clsx** — Conditional classname utility
- **Puppeteer** — Headless browser (chart rendering)

9. Security Architecture

Authentication Flow

Login (Email/Google/Apple) → **Turnstile** (Bot protection) → **JWT Issue** (HS256, 15min) → **Bearer Token** (Auth header) → **Role Check** (admin/user)

LAYER	IMPLEMENTATION
Password Hashing	bcrypt via passlib (auto-configured cost factor)
JWT Tokens	HS256, 15-min access / 7-day refresh, python-jose library
OAuth Validation	Google: ID token sig via public keys. Apple: JWKS with 1hr cache
Bot Protection	Cloudflare Turnstile on registration endpoint
CORS	Whitelist: rigacap.com, www.rigacap.com, localhost dev ports
Admin Guard	Role check + ADMIN_EMAILS allowlist (double enforcement)
Subscription Gate	`require_valid_subscription` dependency on signal endpoints
Payment Security	Stripe Checkout (PCI DSS), webhook signature verification
S3 Access	Price data bucket: all public access blocked, IAM-only
HTTPS	ACM certificates, CloudFront redirect HTTP→HTTPS
Email Cancel Links	JWT-signed one-click cancel URLs for social post management

10. CI/CD Pipeline

Fully automated deployment via GitHub Actions on push to `main`. Zero-downtime updates via Lambda container image swaps.

```
# Trigger: push to main OR pull request to main

Stage 1: Test
  Backend: Python 3.11 → pip install → pytest -v
  Frontend: Node 18 → npm ci → npm run lint → npm test

Stage 2: Deploy (only on main push)
  1. npm run build → S3 sync → CloudFront invalidation
  2. Pre-deploy: export price data to S3
  3. Docker build (linux/amd64, --provenance=false) → ECR push
  4. Lambda update-function-code → API + Worker (same image)
  5. Post-deploy: /api/warmup (verify cold start healthy)
```

> **Critical Lambda Build Requirement:** Docker BuildKit creates multi-platform manifests with attestations that Lambda rejects. All builds use `--provenance=false --sbom=false` flags for compatibility.

11. Monitoring & Performance

Current Monitoring

COMPONENT	METHOD	DETAILS
Lambda Logs	CloudWatch Logs	Python logging, 30-day retention
API Gateway	CloudWatch Metrics	4xx, 5xx error rates
CloudWatch Alarms (10)	SNS → Email	Lambda errors/throttles/duration, API 5xx/4xx, RDS CPU/storage/connections, Worker errors/duration
Pipeline Health Monitor	HealthMonitorService	17 checks across 5 categories: Data Freshness, Worker Performance, Database, Signals, Infrastructure
Data Fetch Tracking	S3 metadata (JSON)	Source used (Alpaca/yfinance), settlement result, fetch duration, fallback status
Database	RDS console metrics	CPU, storage, connections

Performance Characteristics

OPERATION	LATENCY	RESOURCE
Dashboard load	<2 seconds	Pre-computed JSON from CDN
Signal scan (7,381 symbols)	30–45 seconds	Alpaca SIP download + calculation
Single backtest (90d, 100 symbols)	20–40 seconds	Worker Lambda 3GB memory
Walk-forward (5yr, 26 periods)	10–30 minutes	Step Functions → Lambda
AI content generation	2–5 seconds	Claude API (Sonnet 4.5)
Chart card generation	1–3 seconds	matplotlib (Worker Lambda)

Resource Utilization

1 + 3 GB	~800 MB	20 GB	~179 MB
API + WORKER MEMORY	TYPICAL USAGE	RDS STORAGE	S3 PICKLE (GZIP)

12. Dependencies

Backend (Python 3.11)

- **FastAPI** 0.109.0 + Uvicorn 0.27.0
- **SQLAlchemy** 2.0.25 + asyncpg 0.29.0
- **Pydantic** 2.5.0
- **pandas** 2.1.4 + numpy 1.26.3
- **alpaca-py** ≥0.30.0 (SIP market data)
- **yfinance** ≥1.1.0 (fallback + index data)
- **Optuna** ≥3.5.0
- **matplotlib** ≥3.8.0
- **boto3** 1.34.0 + Mangum 0.17.0
- **Stripe** 7.10.0
- **python-jose** 3.3.0 + passlib 1.7.4
- **APScheduler** 3.10.4 (local dev; EventBridge in prod)
- **httpx** 0.26.0 (Claude API client)

Frontend (Node 18)

- **React** 18.2.0
- **Vite** 5.0.0
- **TailwindCSS** 3.3.5
- **React Router** 6.20.0
- **Recharts** 2.10.0
- **Axios** 1.6.0
- **Lucide React** 0.292.0
- **Puppeteer** 24.37.3
- **ESLint** 8.53.0
- **Vitest** 0.34.6

RigaCap

Technical Architecture Document — v3.0.0 — April 2026

This document is confidential and intended solely for authorized technical reviewers.
Infrastructure details should not be shared outside the organization.